

《数字电路设计》 实验报告

实验一 组合逻辑电路实验	2
1.1 实验目的	2
1.2 实验步骤	2
1.3 实验结果展示	5
1.4 实验结果分析	5
1.5 实验总结与心得体会	5
1.6 实验不足与改进措施	6
实验二 译码器和计数器设计	7
2.1 实验目的	7
2.2 实验步骤	7
2.3 实验结果展示	10
2.4 实验结果分析	10
2.5 实验总结与心得体会	10
2.6 实验不足与改进措施	10
实验三 同步时序逻辑电路实验	11
3.1 实验目的	11
3.2 实验步骤	11
3.3 实验结果展示	12
3.4 实验结果分析	12
3.5 实验总结与心得体会	12
3.6 实验不足与改进措施	13

刘鼎琨
2023302121009

实验一 组合逻辑电路实验

1.1 实验目的

- (1) 学习使用 NEXYS4 A7-100T 口袋版。
- (2) 熟悉使用 Vivado 开发软件的基本功能。
- (3) 熟练掌握组合逻辑电路的设计方法，及利用卡诺图简化等。

1.2 实验步骤

•【任务一】

在举重比赛中，有两名裁判员，一名主裁判。当两名以上裁判（必须包括主裁判在内）认为运动员上举杠铃合格，按动电钮，裁决合格信号灯亮，则该运动员成绩有效。

任务要求如下：

1. 用下方的滑动开关做为输入，滑动开关上方的 LED 灯做为输出。
 2. 用 Verilog 的逻辑语句实现门电路逻辑功能。比如一个与门 $F=AB$ 。
- 用逻辑语句实现的基本语法是 $\text{and}(F,A,B)$ ，尽量使用与非门和非门完成实验。

(1) 实验设计思路：

第一次实验：

任务一. 举重比赛

① 列真值表：A为主裁判，F为输出

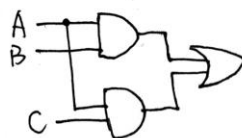
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

② $F = \sum m(5, 6, 7)$

F	AB	00	01	11	10
0	0	0	0	1	0
1	0	0	0	1	1

$$F = AB + AC$$

③ 逻辑图



注：若尽可能使用“与”和“非”门完成 $\overline{F} = \overline{AB} \overline{AC}$

图 1.1 任务 1 实验设计思路

(2) 实验源代码：

- 不考虑使用与非门的要求，可得到简化代码：

```
module test( input [15:0] sw_i, output [15:0] led_o);
begin
    assign led_o[0] = (sw_i[0] & sw_i[1]) | (sw_i[0] & sw_i[2]);
end
endmodule
```

- 考虑器材的要求，给出只利用与非门的相关代码：

```

module test( input [15:0] sw_i, output [15:0] led_o);
begin
    wire tmp1, tmp2;
    nadd(sw_i[0], sw_i[1], tmp1);
    nadd(sw_i[0], sw_i[2], tmp2);
    nadd(tmp1, tmp2, led_o[0]);
end
endmodule

```

```

module nadd( input a, input b, output c);
begin
    assign c = ~(a & b);
end
endmodule

```

•【任务二】

设计一个多输出组合逻辑电路。该电路的输入是一个 BCD 码。

输入为 4 位 BCD 码，当电路检测到输入的代码大于 4 时，电路的输出 $F_1=1$ ，其它情况 $F_1=0$ 。当输入的代码小于 6 时，电路的另一个输出 $F_2=1$ ，其它情况 $F_2=0$ 。

(1) 实验设计思路：

任务二：双输出

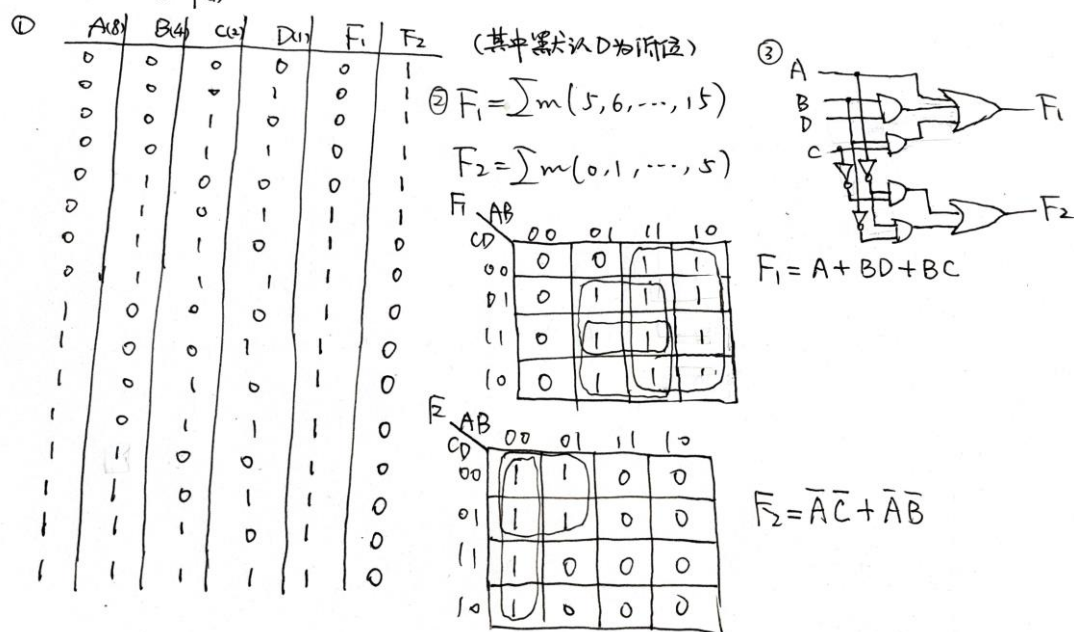


图 1.2 任务 2 实验设计思路

(2) 实验源代码:

```
module test(input [15:0] sw_i, output [15:0] led_o);
    wire t1, t2, t3, t4, t5, t6;
    and(t1, sw_i[2], sw_i[0]);
    and(t2, sw_i[2], sw_i[1]);
    or(led_o[1], sw_i[3], t1, t2);
    and(t3, ~sw_i[3], ~sw_i[1]);
    and(t4, ~sw_i[3], ~sw_i[2]);
    or(led_o[0], t3, t4);
endmodule
```

也可以使用 if 语句、case 语句等行为级描述完成相应功能。

• 【任务三】

熟悉 vivado 的开发环境，完成手册相关任务。

基本的操作已完成，这里总结几点：

1. 如何在“综合”步骤中找到真值表

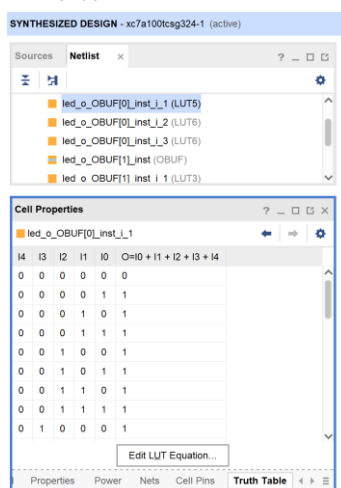


图 1.3 在“综合”中找到真值表

2. 如何使用功能仿真：根据文档中的介绍，可以使用手写.tb 文件的方式操作，也可以用 Vivado 自带的仿真输出相关 file。

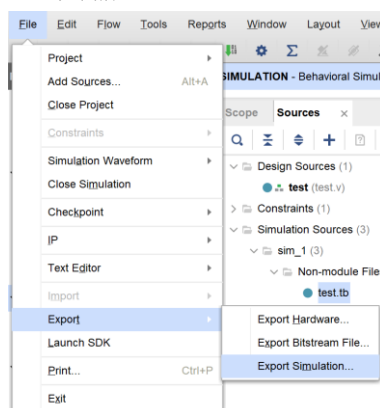


图 1.4 一种可能的仿真文件输出方式

1.3 实验结果展示

- 【任务一】

说明：最右侧的拨码开关代表主裁判，前两个为副裁判。灯亮为通过。下面选择三种比较典型的情况。



图 1.5 任务一的三种典型输出

- 【任务二】

说明：拨码开关左侧为高位， f_1 对应 LED0， f_2 对应 LED1。



图 1.6 任务二的两种典型输出

1.4 实验结果分析

通过遍历真值表的全部情况，发现结论和设计要求保持一致，较好地完成了相关任务。

1.5 实验总结与心得体会

1. 实验总结：本次实验最主要的成就是成功配置了 Vivado 软件，感谢实验室老师提供的相关 setup。此外，对两个基本的组合逻辑问题，我验证、熟练使用了 Vivado 的基本功能，为后期更为复杂的实验打好了相关基础。

2. 心得体会：对于实验中出现的問題等，我们应该及时解决，比如字体大小的问题、编程环境的问题，如果实现配置好相关的高级功能，肯定能让自己的实验速度大大提升，下附两个相关的链接：

- 字体调节：https://blog.csdn.net/weixin_62432906/article/details/130811454
- 和 VSCode 关联：https://blog.csdn.net/Xie_Tao_1995/article/details/128219080
- VSCode 相关配置：<https://blog.csdn.net/w18864443115/article/details/135604467>

1.6 实验不足与改进措施

有一处代码问题的检查花费了较多的时间，具体是写错了一个变量的输入输出类型。

改进措施：要知道在“综合”环节时，只会检查代码中的基础语法错误，无法判定逻辑错误。况且 FPGA 相关的编程软件，包括 Quartus 等，在编译、布线等工作的耗时很长，如果代码需要经常修改调试等，会比较浪费时间，最好在写的时候就集中精力，不要把可能存在的问题都拖给 Vivado。

实验二 译码器和计数器设计

2.1 实验目的

- (1) 学习并实践 NEXYS4 A7-100T 口袋版时钟分频功能。
- (2) 学习并实现译码器和计数器的设计方法

2.2 实验步骤

•【任务一】
使用 Verilog 语言设计一个 BCD/七段译码器。实验要求如下：
1. 用下方的滑动开关做为输入，滑动开关上方的七段数码管做为输出。
2. 输入要求为 0-9 的 BCD 码，转为相应的十进制数字显示，输入大于 9 的数字则七段数码管全灭。

(1) 实验设计思路：

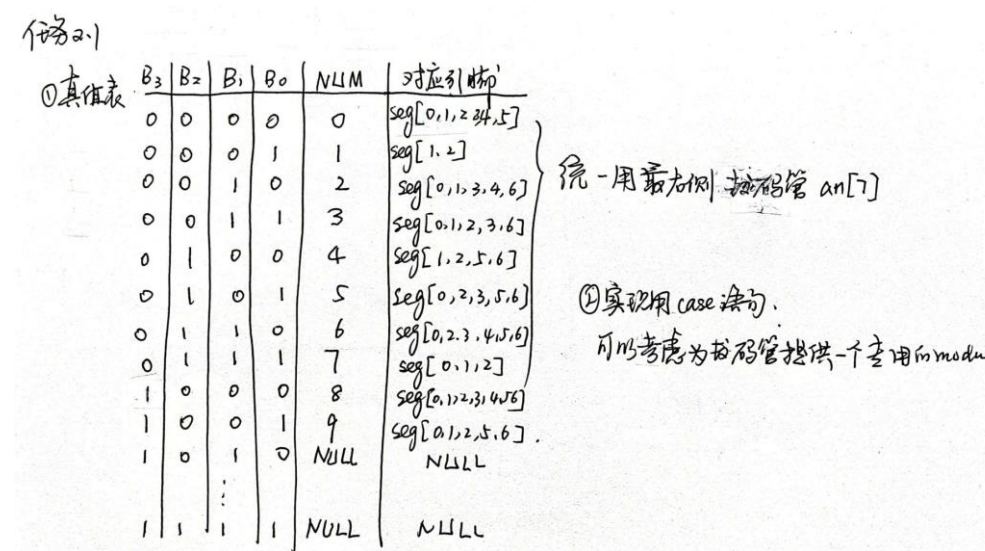


图 2.1 任务 1 实验设计思路

显示字符	共阳极段选码	显示字符	共阳极段选码
0	C0H	A	88H
1	F9H	B	83H
2	A4H	C	C6H
3	B0H	D	A1H
4	99H	E	86H
5	92H	F	84H
6	82H	全黑	FFH
7	F8H		
8	80H		
9	90H		

图 2.2 七段式数码管的共阳极编码

(2) 实验源代码:

```
module test01(
    input clk,
    input [15:0]sw_i,
    output reg[7:0]disp_an_o = 8'b1111_1110,
    output reg[7:0]disp_seg_o
);
// mode check
always@(posedge clk)
begin
    if(sw_i[3:0] > 9) disp_an_o[0] <= 1'b1;
    else disp_an_o[0] <= 1'b0;
end

//print
always@(*)
case(sw_i[3:0])
    0:disp_seg_o<=8'hC0;
    1:disp_seg_o<=8'hF9;
    2:disp_seg_o<=8'hA4;
    3:disp_seg_o<=8'hB0;
    4:disp_seg_o<=8'h99;
    5:disp_seg_o<=8'h92;
    6:disp_seg_o<=8'h82;
    7:disp_seg_o<=8'hF8;
    8:disp_seg_o<=8'h80;
    9:disp_seg_o<=8'h90;
    default: disp_seg_o<=8'hC0;
endcase
endmodule
```

•【任务二】

设计一个十进制计数器，用七段数码管显示计数值。计数允许 EN 及清零 RST 信号。实验要求如下：从 0-9 循环计数，按下 RST 信号则从 0 开始重新计数。

(1) 实验设计思路:

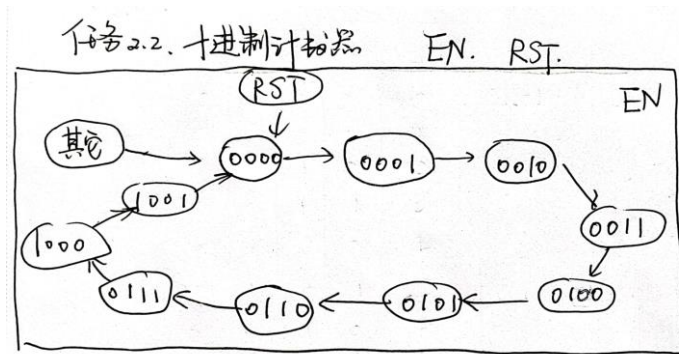


图 2.3 任务 2 实验设计思路

(2) 实验源代码:

```

module test01(
    input clk,
    input rstn,
    input [15:0]sw_i,
    output reg[7:0]disp_an_o =
    8'b1111_1110,
    output reg[7:0]disp_seg_o
);

// division!
parameter div_num=24;
wire clk_divnum;
reg[31:0]clk_cnt;
always@(posedge clk)
begin
    clk_cnt<=clk_cnt+1'b1;
end
assign clk_divnum=clk_cnt[div_num];

//conuter!
reg[3:0] Q = 4'b0000;
always@ (posedge clk_divnum)
begin
    if(sw_i[0])
    begin
        disp_an_o[0] <= 1'b0;
        if(Q == 4'b1001|| !rstn)
        Q <= 4'b0000;
        else
        Q <= Q + 4'b0001;
    end
    else
    begin
        disp_an_o[0] <= 1'b1;
        Q <= 4'b1001;
    end
end

//print
always@(*)
case(Q)
    4'b0000:disp_seg_o<=8'hC0;
    4'b0001:disp_seg_o<=8'hF9;
    4'b0010:disp_seg_o<=8'hA4;
    4'b0011:disp_seg_o<=8'hB0;
    4'b0100:disp_seg_o<=8'h99;
    4'b0101:disp_seg_o<=8'h92;
    4'b0110:disp_seg_o<=8'h82;
    4'b0111:disp_seg_o<=8'hF8;
    4'b1000:disp_seg_o<=8'h80;
    4'b1001:disp_seg_o<=8'h90;
    default:disp_seg_o<=8'hC0;
endcase
endmodule

```

2.3 实验结果展示

- 【任务一】

说明：三幅图分别对应临界状态的输出。

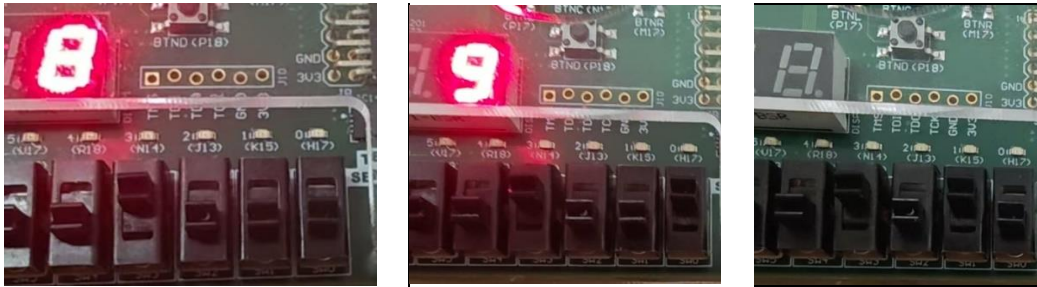


图 2.4 任务一的三种典型输出

- 【任务二】 由于涉及动态过程，不另附图.

2.4 实验结果分析

任务一，能够完成输入和输出的对应，在大于 9 的情况下，实现了熄灭处理.

任务二，能够完成 10 进制技术功能，具体实现为同步时序逻辑，利用不同 always 语句串联而成，同时能够实现 RES 置零，EN 清屏及置零.

2.5 实验总结与心得体会

1. 实验总结：学习了基本的分频方法和计数器原理，同时能够灵活的使用数码管.
 - ① `output reg[7:0]disp_an_o = 8'b1111_1110`，这段代码可以放置于 module 参数声明的过程中，非常便捷. 如果不在此时进行赋值，module 内部需要利用 initial，或者 always，前者根本不能烧录到板子上.
 - ② 除了定义时，位数标在前面，其余时候位数标在后面.
2. 心得体会：我对于时序逻辑的认识更加深刻，也意识到只要我下放到门级电路，直接使用状态参量进行行为级描述，难度也并不是很大。同时，感觉很多的不确定因素来源于对并行机制了解的不够透彻.

2.6 实验不足与改进措施

很遗憾没有实现多位数码管显示进位，调试无果还发生了系统错误，下次有机会可以再尝试一下.

实验三 同步时序逻辑电路实验

3.1 实验目的

- (1) 学习并实践 Verilog 中 D 触发器功能。
- (2) 学习并实现同步时序逻辑电路

3.2 实验步骤

•【任务一】

使用 Verilog 语言定义 D 触发器，D 触发器的时钟信号使用开发板上的“BTNC”按钮做为单步脉冲，并使用 D 触发器设计一个可重叠同步时序逻辑电路。实验要求如下：任务要求如下：

(转专业班)设计一个可重叠的“101”序列检测器,当输入“101”时,输出 1。

- (1) 实验设计思路:

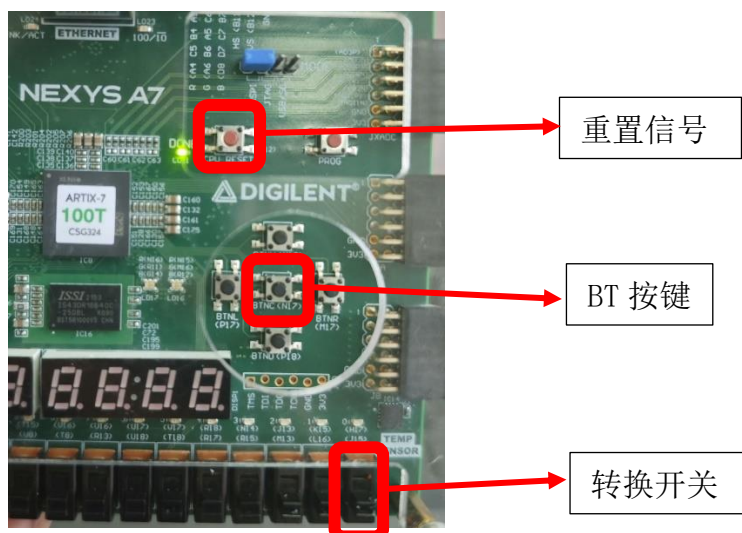


图 3.1 任务 1 的按键设计

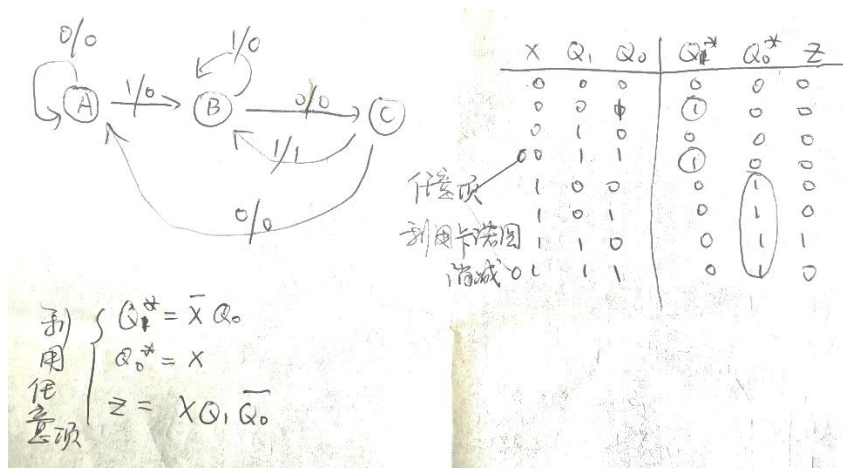


图 3.2 任务 1 实验设计思路

(2) 实验源代码:

```
module DFF(d, cp, Q, rst);
    output reg Q;
    input d, cp, rst;
    always@(posedge cp or negedge rst)
    begin
        if(!rst) Q <= 1'b0;
        else Q <= d;
    end
endmodule
```

```
module test01(
    input rstn,
    input BTNC,
    input [15:0]sw_i,
    output [15:0]led_o
);
    wire q0, q1, q2;
    wire d0, d1, d2;
    DFF (d0, BTNC, q0, rstn);
    DFF (d1, BTNC, q1, rstn);
    DFF (d2, BTNC, q2, rstn);
    assign d0 = sw_i[0];
    assign d1 = ~sw_i[0] & q0;
    assign d2 = sw_i[0] & q1 & ~q0;
    assign led_o[0] = q2;

endmodule
```

3.3 实验结果展示

- 【任务一】

动态过程不方便放置视频，此处不进行结果的放置，已当堂完成验收。

3.4 实验结果分析

实验结果吻合基本预设，能完成可重叠的“101”序列检测。

3.5 实验总结与心得体会

1. 实验总结：本次实验发现了很多的问题，首先是自己设计的 `always` 块内出现了不转换的情况，原因是没有把寄存器的输出反输入到表达式中。后来是代码逻辑正常，但是需要按键五次才能实现相应的效果，这个问题可能是由于时序逻辑有漏洞造成的。

2. 心得体会：本次实验，我实践了课堂教学中关于时序逻辑设计的部分，并用 verilog 代码进行实现。最重要的是我体会到了代码算法对于实验正确性的影响，对于硬件描述语言，不同时序、不同模块之间都会有各种接口问题，如果一个算法可以减少模块数量，牺牲一些空间也是可接受的。

本实验中，`assign d2 = sw_i[0] & q1 & ~q0;`

`assign led_o[0] = q2;`

尤为重要，相当于间接利用了第三个 D 触发器的 BTNC 端进行了时序控制！避免在主模块中引入 `always` 语句。

3.6 实验不足与改进措施

未能最终修改成功自己开始写的一种思路，时序上总有各种问题，在这里附上这个“失败”的代码，期待有人有时间能找到问题：

```
module DFF(
    input CP,
    input Din,
    input rstn,
    output reg Dout);
    always @(posedge CP or negedge rstn)
    begin
        if(!rstn)
        begin
            Dout <= 0;
        end
        else
        begin
            Dout <= Din;
        end
    end
end
endmodule

module test01(
    input BTNC,
    input rstn,
    input [15:0]sw_i,
    output reg[7:0]disp_an_o = 8'b1111_1110,
    output reg[7:0]disp_seg_o
);
    wire [1:0] toQ;
    reg [1:0] Q;
    reg [1:0] D;
    reg light;
```

```

//main logic
DFF dff1(.CP(BTNC), .Din(D[1]), .rstn(rstn), .Dout(toQ[1]));
DFF dff0(.CP(BTNC), .Din(D[0]), .rstn(rstn), .Dout(toQ[0]));

always @(posedge BTNC or negedge rstn)
begin
    if (!rstn)
    begin
        Q <= 2'b00;
        D <= 2'b00;
        disp_an_o[0] <= 1'b1;
    end
    else
    begin
        D[1] <= (~Q[1] & Q[0] & sw_i[0]) | (Q[1] & ~Q[0] & sw_i[0]);
        D[0] <= ~Q[1] & ~Q[0] & sw_i[0];
        light <= toQ[1] & ~toQ[0] & sw_i[0];
        disp_an_o[0] <= 0'b0;
    end
end

//print
always @(*)
begin
    case(light)
        0: disp_seg_o <= 8'hC0;
        1: disp_seg_o <= 8'hF9;
        default: disp_seg_o <= 8'hC0;
    endcase
end

endmodule

```